

DISEÑO DE SOFTWARE · UCOM0310

Defensa de la Ae1 — Diseño orientado a objetos

Guía de apoyo para justificar el diseño de GesTutoUEES: decisiones clave y respuestas a las preguntas probables.

1 Las 4 decisiones que sostienen el diseño

Herencia solo donde es verdad

`Estudiante` y `Docente` heredan de `Usuario` (abstracta) porque *son* usuarios, no para reutilizar código. Nadie se registra como "usuario genérico". Si mañana aparece un `Coordinador`, hereda y el resto no se entera.

Composición para lo demás

Una `Reserva` *tiene* un `Estudiante` y un `Horario`; no hereda de ellos. Heredar ahí sería forzar la relación. La composición modela el "tiene-un", la herencia el "es-un".

El estado se protege, no se expone

`Horario` no deja cambiar `disponible` desde fuera: solo `ocupar()` y `liberar()`. La regla anti doble reserva vive en un solo lugar y no se puede saltar por accidente.

El dominio depende de abstracciones

`ServicioReservas` habla con las interfaces `Notificador` y `RepositorioReservas`, no con tecnologías concretas. Cambiar correo por SMS o memoria por BD toca una clase nueva, no el dominio.

2 Preguntas probables • Diseño OO y UML

¿Por qué Usuario es abstracta y no una interfaz?

Porque Estudiante y Docente comparten **estado y comportamiento** (id, nombre, correo). Una interfaz no puede tener estado. La herencia aquí es una relación "es-un" real, no un truco para reutilizar código.

¿Dónde está la regla anti doble reserva y por qué ahí?

En `Horario.ocupar()` : lanza excepción si ya está ocupado. Está ahí porque **el dueño del dato es quien lo protege**. Si la regla viviera en el servicio, cualquier otro código podría ocupar el horario saltándose la validación.

¿Qué relación hay entre Reserva, Estudiante y Horario?

Asociación por composición: la reserva *tiene* un estudiante y un horario. En el UML se dibuja con líneas de asociación; en el código son atributos. No es herencia porque una reserva no "es" un estudiante.

¿Cómo se garantiza que el UML y el código no se contradigan?

El UML se generó del código (PlantUML en `docs/modelo-clases.puml`). Lo que está en uno está en el otro: mismas clases, misma herencia, mismas interfaces y mismas asociaciones.

3 Preguntas probables • SOLID, cohesión y acoplamiento

¿Dónde se ve el DIP (Dependency Inversion)? la más probable

En el constructor de `ServicioReservas` : recibe `RepositorioReservas` y `Notificador` (abstracciones), no `RepositorioReservasEnMemoria` ni `NotificadorCorreo` (concretas). Las implementaciones se inyectan en `Main` . Si cambio la persistencia, el servicio no se modifica.

¿Y el OCP (Open/Closed)?

Puedo agregar un `NotificadorSms` o un `RepositorioReservasPostgres` implementando las interfaces, **sin tocar** `ServicioReservas` . El sistema está abierto a extensión y cerrado a modificación.

¿Cómo justificas la cohesión alta?

Cada clase hace una cosa: `Horario` maneja su tiempo, `Reserva` su ciclo de vida, `ServicioReservas` coordina. Si algo falla en la reserva, sé exactamente a qué clase ir. Eso es cohesión.

¿Cómo controlaste el acoplamiento?

Con las dos interfaces. `ServicioReservas` no sabe si el aviso sale por correo o WhatsApp, ni si los datos van a memoria o a una BD. El acoplamiento queda en las abstracciones, no en las tecnologías.

4 Si me preguntan "¿por qué este diseño es sólido?"

Respuesta corta (la que resume todo)

"Un sistema no es sólido por lo complejo que es, sino por lo **barato que le resulta el próximo cambio**. Aquí, cambiar la tecnología de aviso o de persistencia toca una clase nueva, no el dominio. Y la regla más importante —que un horario ocupado no se reserve dos veces— está protegida dentro del propio objeto, no repartida por el código."

¿Qué fue lo más difícil de decidir?

Separar el dominio de las tecnologías. La tentación era que `ServicioReservas` creara el `NotificadorCorreo` por dentro (más rápido), pero eso ataba el dominio a una tecnología. Lo aislé con la interfaz `Notificador` y la inyección por constructor.

¿Qué mejorarías si tuvieras más tiempo?

Agregaría la reprogramación como una operación de primer nivel en el servicio (hoy la reserva la soporta, pero el servicio no la expone), y pruebas unitarias para la regla anti doble reserva y el ciclo de vida de la reserva.